

IAM Tasks

1. Create one IAM user and assign EC2 and S3 full access roles.



Task: Create IAM User with EC2 & S3 Full Access

Step 1: Open IAM

1. Log in to the **AWS Management Console**.
2. In the search bar, type **IAM**.
3. Open **IAM** → **Users**.
4. Click **Create user**.

Step 2: Create the User

Enter a user name, for example:

- **IAM-Admin**
- Click **Next**.

Step 3: Assign Permissions

- Select **Attach policies directly**.
- Search for and select:
 - **AmazonEC2FullAccess** → Full access to EC2
 - **AmazonS3FullAccess** → Full access to S3
- Click **Next** → **Create user**.

Policy Purpose

- **AmazonEC2FullAccess** → Grants full access to manage EC2 instances.
- **AmazonS3FullAccess** → Grants full access to manage S3 buckets and objects.

Evidence

- After creation, go to **IAM** → **Users** → **IAM-Admin** → **Permissions tab**.
- You will see the attached policies listed.
- Capture a screenshot of this summary page showing the policies.

The screenshot shows the 'Retrieve password' step in the AWS IAM console. A green banner at the top states 'User created successfully' and provides a 'View user' link. On the left, a progress bar shows four steps: 'Specify user details', 'Set permissions', 'Review and create', and 'Retrieve password' (which is the current step). The main content area is titled 'Retrieve password' and includes a 'Console sign-in details' box. This box contains the 'Console sign-in URL' (https://986881030460.signin.aws.amazon.com/console), the 'User name' (chandu), and the 'Console password' (masked with asterisks and a 'Show' link). At the bottom right, there are three buttons: 'Cancel', 'Download .csv file', and 'Return to users list'.

The screenshot shows the 'IAM users' list in the AWS IAM console. The left sidebar shows the 'Identity and Access Management (IAM)' menu with options like 'Dashboard', 'Roles', 'Policies', 'IAM users', 'IAM user groups', 'Identity providers', 'Account settings', 'Root access management', and 'Temporary delegation requests'. The main content area is titled 'IAM users (1)' and shows a table with one user, 'chandu'. The table columns are: 'User name', 'Path', 'Group', 'Last activity', 'MFA', 'Password age', 'Console last sign-in', and 'Access key ID'. The 'chandu' user is listed with a path of '/', group of '0', last activity of '15 minutes ago', MFA status of '-', password age of '14 minutes', console last sign-in of '15 minutes ago', and no access key ID. At the top right, there are 'Delete user' and 'Create user' buttons.

The screenshot shows the 'chandu' user details page in the AWS IAM console. The left sidebar is the same as the previous screenshot. The main content area is titled 'chandu' and includes a 'Summary' section with the user's ARN (arn:aws:iam::986881030460:user/chandu), creation date (September 15, 2026, 15:28 (UTC+05:30)), console access status (Enabled without MFA), and last console sign-in (Today). There is also a link to 'Access key 1' and a 'Create access key' button. Below the summary, there are tabs for 'Permissions', 'Groups', 'Tags', 'Security credentials', and 'Last Accessed'. The 'Permissions' tab is selected, showing 'Permissions policies (3)'. These policies are 'AmazonEC2FullAccess' and 'AmazonS3FullAccess', both of which are 'AWS managed' and 'Attached via Directly'. At the top right, there is a 'Delete' button. At the bottom right, there are buttons for 'Simulate', 'Remove', and 'Add permissions'.

2. Create one group in IAM and assign read access for EC2.



Task 2: Create IAM Group with EC2 Read Access

Step 1: Open IAM

1. Log in to the **AWS Management Console**.
2. In the search bar, type **IAM**.

3. Open **IAM** → **Groups**.
4. Click **Create group**.

Step 2: Create the Group

Enter a group name, for example:

- **Ec2-Readgroup**
- Click **Next**.

Step 3: Assign Permissions

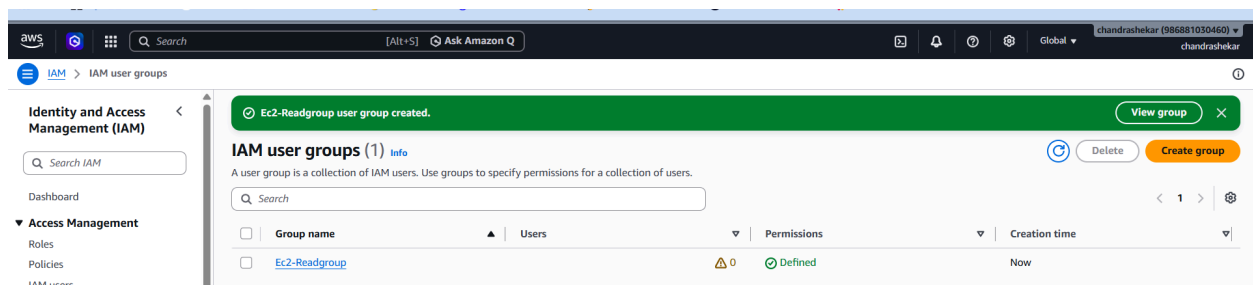
- Search for and select:
 - **AmazonEC2ReadOnlyAccess** → Grants read-only access to EC2 resources.
- Click **Create group**.

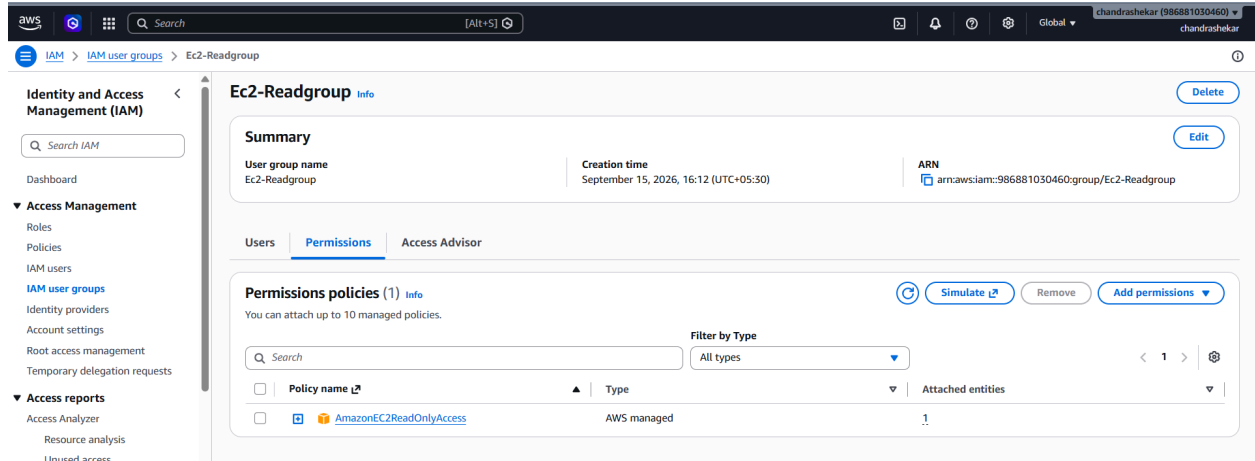
Policy Purpose

- **AmazonEC2ReadOnlyAccess** → Allows viewing EC2 instances, volumes, and configurations, but prevents creating, modifying, or deleting them.

Evidence

- After creation, go to **IAM** → **Groups** → **Ec2-Readgroup** → **Permissions tab**.
- You will see the attached policy listed.
- Capture a screenshot of the group summary showing the policy and group details.





3. Create a new user named "Devops" and add to the group created in task 2.



Task 3: Create IAM User “Devops” and Add to Group

Step 1: Open IAM

1. Log in to the **AWS Management Console**.
2. In the search bar, type **IAM**.
3. Open **IAM** → **Users**.
4. Click **Create user**.

Step 2: Create the User

Enter the user name:

- **Devops**
- Click **Next**.

Step 3: Assign to Group

- On the permissions page, choose **Add user to group**.

Select the group created in Task 2:

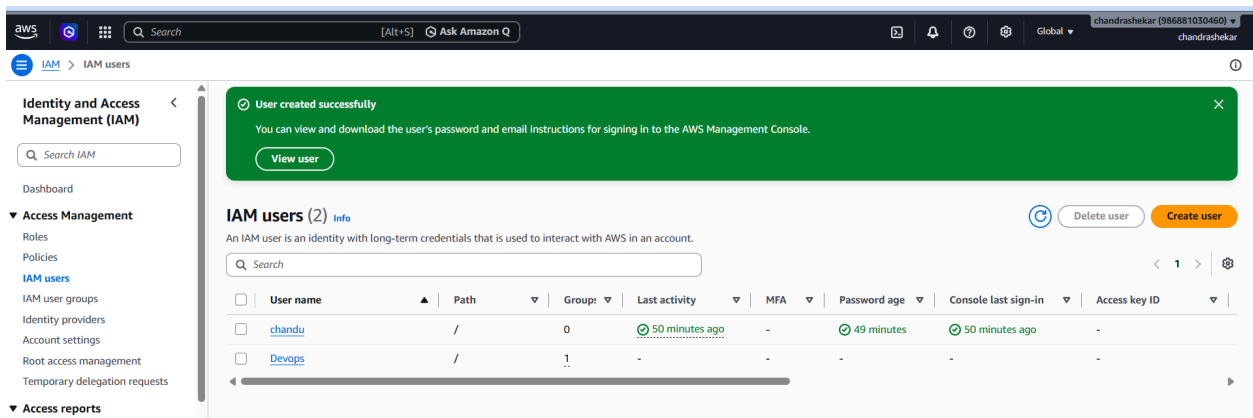
- **Ec2-Readgroup**
- Click **Next** → **Create user**.

Policy Purpose

- By adding the **Devops** user to **Ec2-Readgroup**, the user inherits the **AmazonEC2ReadOnlyAccess** policy.
- This means **Devops** can view EC2 resources but cannot create, modify, or delete them.

Evidence

- Go to **IAM** → **Users** → **Devops** → **Groups tab**.
- Confirm that **Ec2-Readgroup** is listed.
- Capture a screenshot showing the user's group membership and inherited permissions.



4. Write a bash script to create an IAM user with VPC full access.

01 Write the Bash Script

Create a script that defines the IAM user and attaches the VPC policy.

File: create_vpc_user.sh

- Add `aws iam create-user --user-name VpcUser`
- Add `aws iam attach-user-policy --user-name VpcUser --policy-arn arn:aws:iam::aws:policy/AmazonVPCFullAccess`
- Save and exit

02 Make Script Executable

Ensure the script can be run from Git Bash.

`chmod +x create_vpc_user.sh`

- Run `chmod +x create_vpc_user.sh`

- Confirm with `ls -l` that it has execute permission

03 Run the Script

Execute the script to create the IAM user and attach the policy.

```
./create_vpc_user.sh
```

- Script output should show user ARN
- Verify with `aws iam list-users`
- Check attached policies with `aws iam list-attached-user-policies --user-name VpcUser`

04 Add Credentials for User

Required for Login

By default, the user has no login profile or access keys. You must add them using an admin account.

- For console login: `aws iam create-login-profile --user-name VpcUser --password 'StrongPassword123!' --password-reset-required`
- For CLI/API: `aws iam create-access-key --user-name VpcUser`
- Configure with `aws configure`

05 Test Permissions

Verify the user can perform VPC actions but not IAM actions.

- Run `aws ec2 describe-vpcs` → should succeed
- Run `aws iam list-users` → should fail (VpcUser has no IAM rights)
- This confirms policy is applied correctly


```
HP@DESKTOP-1GQO65F MINGW64 ~ (master)
$ vi create_vpc_user.sh

HP@DESKTOP-1GQO65F MINGW64 ~ (master)
$ cat create_vpc_user.sh
#!/bin/bash
# Script to create IAM user with VPC Full Access

USER_NAME="VpcUser"

# Step 1: Create IAM user
aws iam create-user --user-name $USER_NAME

# Step 2: Attach AmazonVPCFullAccess policy
aws iam attach-user-policy \
  --user-name $USER_NAME \
  --policy-arn arn:aws:iam::aws:policy/AmazonVPCFullAccess

echo "User $USER_NAME created and granted VPC Full Access"
```

```
HP@DESKTOP-1GQO65F MINGW64 ~ (master)
$ chmod +x create_vpc_user.sh

HP@DESKTOP-1GQO65F MINGW64 ~ (master)
$ ./create_vpc_user.sh
{
  "User": {
    "Path": "/",
    "UserName": "VpcUser",
    "UserId": "AIDAZRK0JXG6ZPQU5QGGP",
    "Arn": "arn:aws:iam::655683598781:user/VpcUser",
    "CreateDate": "2026-09-15T10:58:41+00:00"
  }
}
User VpcUser created and granted VPC Full Access
```

```
HP@DESKTOP-1GQO65F MINGW64 ~ (master)
$ aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "VpcUser",
      "UserId": "AIDAZRK0JXG6ZPQU5QGGP",
      "Arn": "arn:aws:iam::655683598781:user/VpcUser",
      "CreateDate": "2026-09-15T10:58:41+00:00"
    }
  ]
}
```

```

}
User VpcUser created and granted VPC Full Access

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "VpcUser",
      "UserId": "AIDAZRK0JXG6ZPQU5QGGP",
      "Arn": "arn:aws:iam::655683598781:user/VpcUser",
      "CreateDate": "2026-09-15T10:58:41+00:00"
    }
  ]
}

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws iam list-attached-user-policies --user-name VpcUser
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonVPCFullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonVPCFullAccess"
    }
  ]
}

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws configure
AWS Access Key ID [*****ISX7]: AKIA6LRVPE6L44J0VWT
AWS Secret Access Key [*****AoJ9]: lCdC46jw13jYK5jF1004i9eYYX2LFUKKwXRGjH5P
Default region name [us-east-1]: us-east-1
Default output format [json]: json

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$ aws iam list-users
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListUsers operation: User: arn:aws:iam::986881030460:user/chandu is not authorized to perform iam:ListUsers on resource: / because no identity-based policy allows the iam:ListUsers action

HP@DESKTOP-1GQ065F MINGW64 ~ (master)
$

```

5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.

Task 5: Custom IAM Policy for EC2 Access in Specific Regions

Step 1: Define the Policy JSON

Create a file called `EC2RegionPolicy.json` with the following content (example allows EC2 actions only in `us-east-1` and `us-west-2`):

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",

```

```

    "Condition": {
      "StringEquals": {
        "aws:RequestedRegion": [
          "us-east-1",
          "us-west-2"
        ]
      }
    }
  }
]
}

```

Step 2: Create the Policy

Run this command from an **admin/root account**:

```

aws iam create-policy \
  --policy-name EC2RegionRestrictedAccess \
  --policy-document file://EC2RegionPolicy.json

```

Step 3: Attach Policy to User

Attach the new policy to your target user (e.g., **Devops**):

```

aws iam attach-user-policy \

  --user-name Devops \

  --policy-arn arn:aws:iam::<your-account-id>:policy/EC2RegionRestrictedAccess

```

Step 4: Verify

1. Log in as the **Devops** user.
2. Run:

```
aws ec2 describe-instances --region us-east-1
```

```
aws ec2 describe-instances --region ap-south-1
```

Policy Purpose

- Restricts EC2 access to only the regions you specify.
- Prevents accidental or unauthorized EC2 usage in other regions.
- Useful for cost control and compliance.

The screenshot shows the 'Specify permissions' step in the AWS IAM console. The left sidebar indicates 'Step 1: Specify permissions' is active. The main area is titled 'Specify permissions' and includes a sub-header 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' Below this is the 'Policy editor' with a 'JSON' tab selected. The JSON editor contains the following code:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "*",
8       "Condition": {
9         "StringEquals": {
10          "aws:RequestedRegion": [
11            "us-east-1",
12            "us-west-2"
13          ]
14        }
15      }
16    ]
17  }
18 }
```

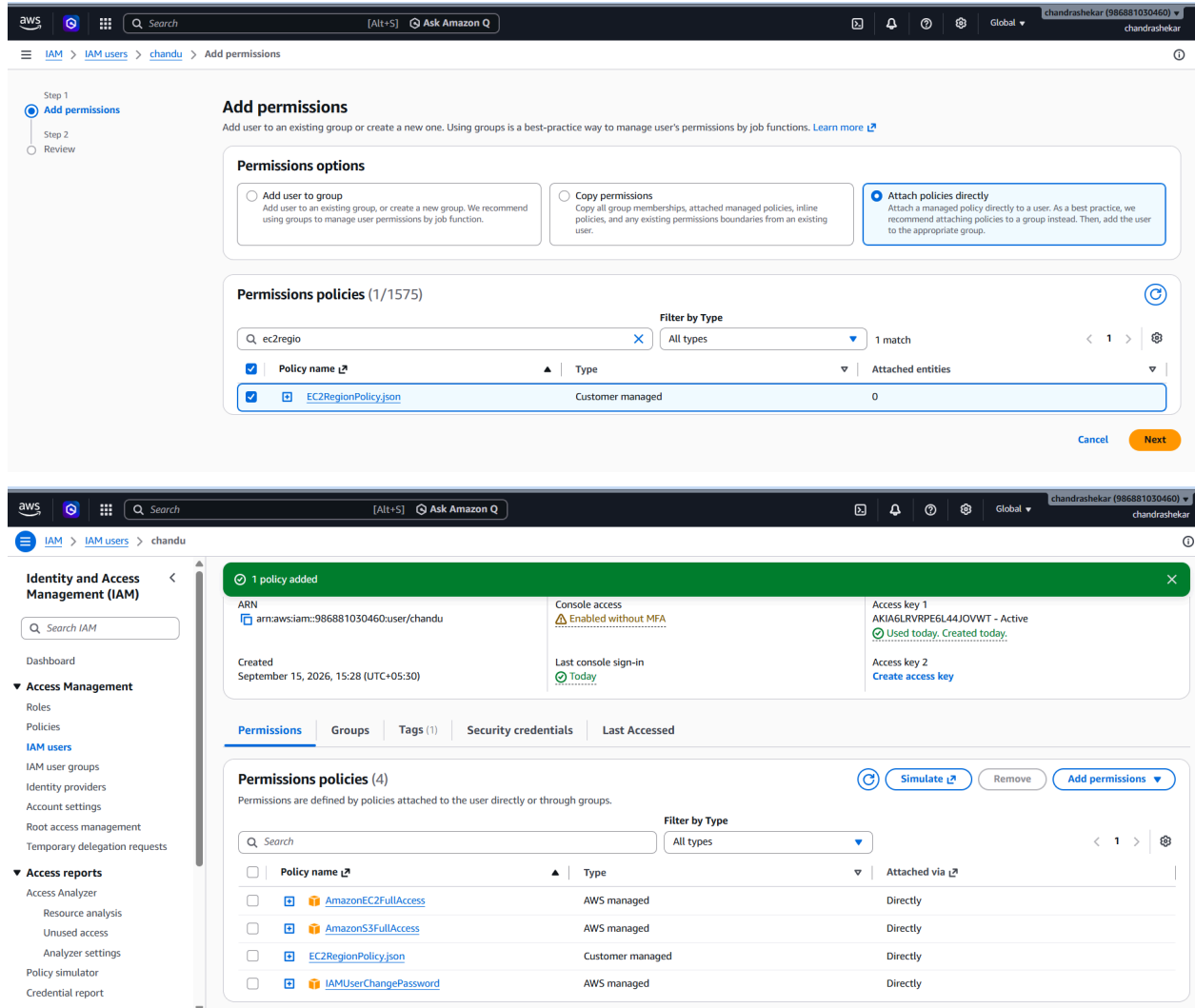
On the right, there is an 'Edit statement' section with a 'Select a statement' prompt and an '+ Add new statement' button.

The screenshot shows the 'Policies' page in the AWS IAM console. A green banner at the top states 'Policy EC2RegionPolicy.json created.' Below this, the 'Policies (1578)' section is visible. A search bar contains 'ec2region'. The 'Filter by Type' dropdown is set to 'All types'. The table below shows a list of policies:

Policy name	Type	Used as
AccessAnalyzerServiceRolePolicy	AWS managed	None
AccountAccessManagerServiceRolePolicy	AWS managed	None

This screenshot shows the same 'Policies' page as the previous one, but with the search filter 'ec2region' applied. The 'Filter by Type' dropdown remains 'All types', and the results now show 1 match:

Policy name	Type	Used as
EC2RegionPolicy.json	Customer managed	None



6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

Task 6: Cross-Account S3 Access (Chandra + Vineeth)

Step 1: Vineeth (Account B: 063298174631) — Create S3 Bucket

- Go to **S3** → **Create bucket**.
- Name: **vineeth-chandra-cross-account-2026**
- Region: **ap-south-1** (Mumbai).
- Upload a test file (e.g., **test.txt**) into the bucket.

Step 2: Chandra (Account A: 986881030460) — IAM User Setup

- IAM user: **Devops**.
- Programmatic access enabled.
- Download access keys.

Configure CLI profile:

bash

aws configure --profile devops

•

Verify:

bash

aws sts get-caller-identity --profile devops

•

👉 Confirms CLI is using your IAM user.

Step 3: Chandra — Attach IAM Policy

Attach inline policy to **Devops** user in Account A:

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::vineeth-chandra-cross-account-2026"
    },
    {
      "Sid": "AllowObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::vineeth-chandra-cross-account-2026/*"
    }
  ]
}
```

Step 4: Vineeth — Add Bucket Policy

In Account B, add this bucket policy:

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowDevopsAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::986881030460:user/Devops"
      },
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": [
        "arn:aws:s3::vineeth-chandra-cross-account-2026",
        "arn:aws:s3::vineeth-chandra-cross-account-2026/*"
      ]
    }
  ]
}
```

👉 This trusts your IAM user ARN from Account A.

Step 5: Chandra — Test Access

Run from CLI:

```
bash
aws s3 ls s3://vineeth-chandra-cross-account-2026 --profile devops --region ap-south-1
```

👉 Should list objects in Vineeth's bucket.

Step 6: Upload & Download Test

Create a file:

```
bash
echo "Cross account S3 test" > test.txt
```

•

Upload:

bash

```
aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region ap-south-1
```

•

Download:

bash

```
aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region ap-south-1
```

•

👉 Confirms full read/write access.

✓ Division of Work

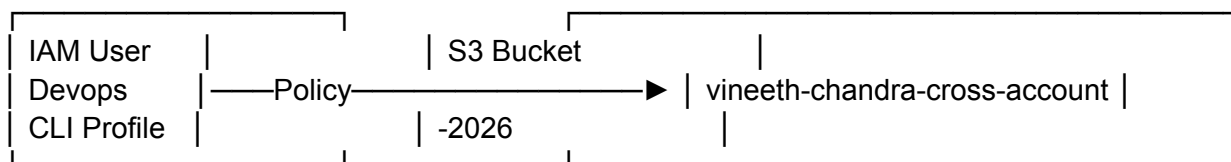
- **Vineeth (Account B)** → Bucket creation + bucket policy.
- **Chandra (Account A)** → IAM user + CLI profile + IAM policy + testing.

📊 Architecture Recap

Code

Account A (986881030460 - Chandra)

Account B (063298174631 - Vineeth)



aws [Search] [Alt+S] Ask Amazon Q Global chandrashekar (986881030460) chandrashekar

Identity and Access Management (IAM) IAM users Devops

Search IAM

Dashboard

▼ Access Management

- Roles
- Policies
- IAM users**
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Policy simulator
- Credential report

Devops Info

Delete

Summary

ARN arn:aws:iam::986881030460:user/Devops	Console access Disabled	Access key 1 AKIA6LRVRPE6LTBLQ6HQ - Active Used today. Created today.
Created September 16, 2026, 15:50 (UTC+05:30)	Last console sign-in -	Access key 2 Create access key

Permissions

Permissions policies (2)

Permissions are defined by policies attached to the user directly or through groups.

Simulate L2 Remove Add permissions

Filter by Type

Search

All types

☐ Policy name L2	Type	Attached via L2
☐ AmazonS3FullAccess	AWS managed	Directly
☐ iam-policy	Customer managed	Directly

```
HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws configure --profile devops
AWS Access Key ID [*****201~]: AKIA6LRVRPE6LTBLQ6HQ
AWS Secret Access Key [*****201~]: JQnWE1GbFu5MXKCIYqwQlBC8K4aq3SpYPbK0rhQ
Default region name [us-east-1]: us-east-1
Default output format [json]: json

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws sts get-caller-identity --profile devops

aws: [ERROR]: Provided region_name 'us-east-1'oesn't match a supported format.

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws sts get-caller-identity --profile devops

aws: [ERROR]: Provided region_name 'us-east-1'oesn't match a supported format.

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws sts get-caller-identity --profile devops

aws: [ERROR]: Provided region_name 'us-east-1'oesn't match a supported format.

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws sts get-caller-identity --profile devops
{
  "UserId": "AIDA6LRVRPE6HMNOYABDX",
  "Account": "986881030460",
  "Arn": "arn:aws:iam::986881030460:user/Devops"
}

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 ls s3://vineeth-chandra-cross-account-2026 --profile devops --region ap-south-1
2026-09-16 17:09:18          96 test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ echo "Cross account S3 test" > test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region ap-south-1
upload: .\test.txt to s3://vineeth-chandra-cross-account-2026/test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region ap-south-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region eu-north-1
upload: .\test.txt to s3://vineeth-chandra-cross-account-2026/test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region eu-north-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt
```

```
HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ echo "Cross account S3 test" > test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region ap-south-1
upload: .\test.txt to s3://vineeth-chandra-cross-account-2026/test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region ap-south-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region eu-north-1
upload: .\test.txt to s3://vineeth-chandra-cross-account-2026/test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region eu-north-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ echo "Cross account S3 test" > test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region eu-north-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region eu-north-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region eu-north-1
upload: .\test.txt to s3://vineeth-chandra-cross-account-2026/test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region eu-north-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws --version
aws-cli/2.36.37 Python/3.14.6 Windows/11 exe/AMD64

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ echo "hi veenth S3 test" > test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp s3://vineeth-chandra-cross-account-2026/test.txt ./ --profile devops --region eu-north-1
download: s3://vineeth-chandra-cross-account-2026/test.txt to .\test.txt

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ aws s3 cp test.txt s3://vineeth-chandra-cross-account-2026/ --profile devops --region eu-north-1
upload: .\test.txt to s3://vineeth-chandra-cross-account-2026/test.txt
```